MULTIMEDI

SPRING / SPRING BOOT TRAINING TRACK

Day 2 – Spring Data JPA & messaging

Persistence, transactions and messaging: how they work, and how they go wrong



DAY 2 – AGENDA

What we will cover today

00	Warm-up: your Day 1 take-home	DISCUSSION
01	Spring Data JPA & the persistence context	EXERCISE
02	Transactions in Spring Data JPA	EXERCISE
03	Data consistency issues, in practice	DISCUSSION
04	Asynchronous integration & transactions	EXERCISE
05	Messaging with Spring AMQP	EXERCISE

DAY 2 • WARM-UP

Your Day 1 take-home: what did you find?

- Refactor one class still using field injection to constructor injection
- Look up a past “no qualifying bean” or duplicate-bean error you've hit
- Check your @PostConstruct/constructor logic, and any missing @PreDestroy
- Check which profile is active by default, and for settings repeated across profiles
- Find a class with several @Value fields and convert it to @ConfigurationProperties
- Search for @Transactional methods called via “this” that might be silently ignored
- Run --debug on one of your own projects and see what the autoconfiguration report shows

TAKE-HOME QUESTION 1 OF 7

Field injection

Refactor one class still using field injection to constructor injection

LET'S DISCUSS

1. Did you find a class with field injection? How many @Autowired fields did it have?
2. What changed when you refactored: tests, readability, or a problem that suddenly surfaced?
3. What would make you use constructor injection by default in new code?

TAKE-HOME QUESTION 2 OF 7

Bean resolution errors

Look up a past “no qualifying bean” or duplicate-bean error you've hit

LET'S DISCUSS

1. Which error was it: no qualifying bean, no unique bean, or a duplicate definition?
2. What was the root cause, and how did you fix it: @Primary, @Qualifier, a missing annotation, a package outside the scan?
3. How did you find out, and what would have made it faster?

TAKE-HOME QUESTION 3 OF 7

Lifecycle logic

Check your @PostConstruct/constructor logic, and any missing @PreDestroy

LET'S DISCUSS

1. Did you find a constructor that does real work: calling a collaborator, doing I/O, starting a thread?
2. Is there a resource that is opened but never closed: a thread pool, a scheduler, a connection, a file?
3. Where would @PostConstruct and @PreDestroy belong in that class?

TAKE-HOME QUESTION 4 OF 7

Profiles

Check which profile is active by default, and for settings repeated across profiles

LET'S DISCUSS

1. Which profile is active by default in your project, and is that intended?
2. Did you find the same value repeated in several profiles? What could move to the shared defaults?
3. What happens in your project when no profile is active?

TAKE-HOME QUESTION 5 OF 7

@ConfigurationProperties

Find a class with several @Value fields and convert it to @ConfigurationProperties

LET'S DISCUSS

1. Which class had several @Value fields? What would the grouped properties class look like?
2. Which of those settings could get a default value or validation?
3. Did the conversion reveal unused or misspelled keys?

TAKE-HOME QUESTION 6 OF 7

Self-invocation

Search for @Transactional methods called via “this” that might be silently ignored

LET'S DISCUSS

1. How many suspicious self-invocations did you find?
2. Could you confirm that a transaction was really missing: a test, logging, a breakpoint?
3. How would you fix it, and would the fix change the design?

TAKE-HOME QUESTION 7 OF 7

Autoconfiguration report

Run --debug on one of your own projects and see what the autoconfiguration report shows

LET'S DISCUSS

1. Which autoconfiguration in the report surprised you?
2. Was something applied that you did not know about, or missing that you expected?
3. Did you spot a candidate to override with your own bean, or to exclude?



DAY 2 • TOPIC 1

Spring Data JPA & the Persistence Context



TOPIC 1 – BASICS

JPA, Hibernate and Spring Data JPA

Your code	services and entities: the part you write
↓	
Spring Data JPA	repositories: you write an interface, Spring writes the implementation
↓	
JPA (Jakarta Persistence)	the standard API: @Entity, EntityManager, queries
↓	
Hibernate	the library that implements JPA, and the default in Spring Boot
↓	
JDBC and the database	the driver and the real tables

JPA is the specification, Hibernate does the work, and Spring Data JPA removes the boilerplate

An entity is a class mapped to a table

CUSTOMER.JAVA

```java
@Entity
@Table(name = "customers")
public class Customer {

  @Id
  @GeneratedValue
  private Long id;

  private String name;
  private String email;

  // constructor, getters and setters
}
```

TABLE: CUSTOMERS

id	name	email
1	Ann	ann@example.org
2	Bob	bob@example.org

- one class = one table
- one object = one row
- one field = one column

JPA needs an @Id and a no-argument constructor (it may be protected)



What a repository gives you

CUSTOMERREPOSITORY.JAVA

```java
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

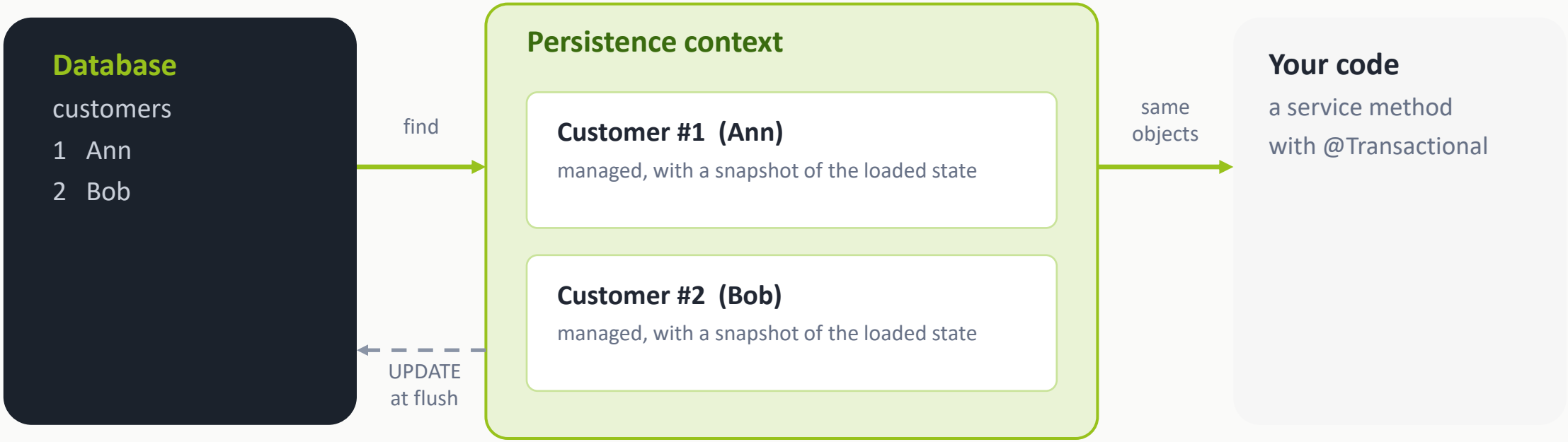
  List<Customer> findByNameContaining(String part);
}
```

- No implementation class: Spring creates it at startup
- It is a proxy, like the transactional proxies of Day 1
- A derived query: the method name becomes the query

YOU GET FOR FREE

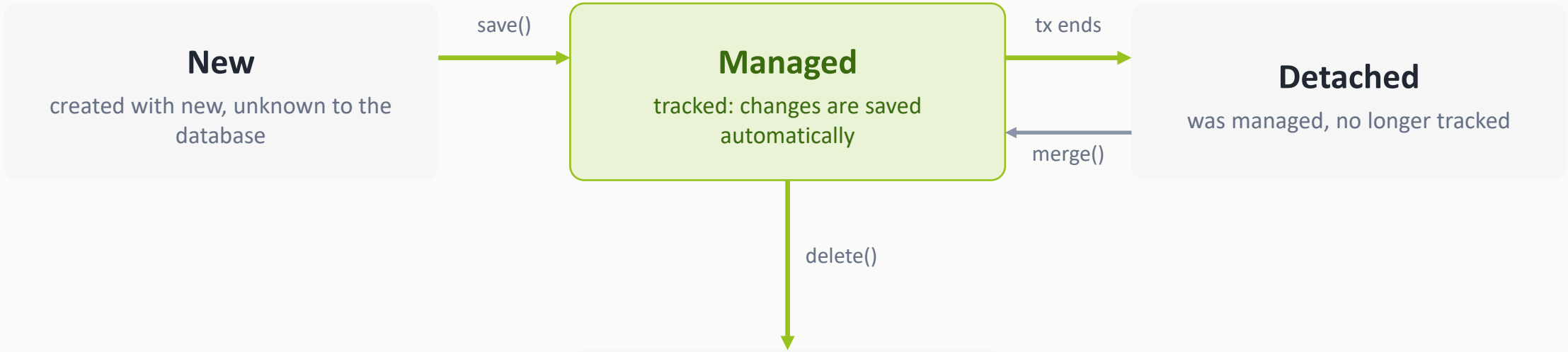
```
save(customer)
findById(id)
findAll()
deleteById(id)
count()
```

The persistence context



- A workspace that tracks every entity you load or save
- One object per row: the same id gives you the same instance
- By default it lives as long as the transaction

The life of an entity: four states



A managed entity becomes detached when the transaction ends, or when the context is cleared

save() on a detached entity merges it: you get a managed copy back

Dirty checking: no save() needed

CUSTOMERSERVICE.JAVA

```java
@Transactional
public void rename(Long id, String newName) {
  Customer c = repository.findById(id).orElseThrow();
  c.setName(newName);    // no save() call
}   // commit: the change is noticed
```

SQL IN THE CONSOLE

```
update customers
set email=?,name=?
where id=?
```

1 Load

Hibernate keeps a snapshot of the loaded state

2 Change

you modify the managed object, nothing is sent yet

3 Commit

Hibernate compares the object with its snapshot and runs an UPDATE if it differs

that is why save() is not needed on a managed entity



TOPIC 1 – WHAT IS IT

Flush is not commit

Transaction begins

a persistence context is opened

You change entities

only in memory so far

Flush

pending SQL is sent to the database

Commit

the changes become permanent and visible to others

WHEN DOES A FLUSH HAPPEN?

Before a commit

always, as part of the commit

Before a query

so the query sees your pending changes (auto flush)

When you ask for it

repository.flush() or saveAndFlush(...)

save() does not mean the SQL has run. A rollback after a flush still undoes everything

Detached entities: changes that go nowhere

THE PROBLEM

```
public Customer find(Long id) {    // no @Transactional
  return repository.findById(id).orElseThrow();
}                                  // now detached

Customer c = service.find(1L);
c.setName("Ann");                 // nobody is watching
// the change never reaches the database
```

Changes are silently lost

no exception, no warning

LazyInitializationException

when you touch a lazy relation after the context closed

The fix

Load, change and finish inside one @Transactional method.

Or call save() on the detached entity: it merges it, and you use the managed copy it returns.

In your daily job: look for code that changes an entity after the method that loaded it has returned

Flush surprises: the error comes later

THE PROBLEM

```
@Transactional
public void changeEmail(Long id, String email) {
  try {
    Customer c = repository.findById(id).orElseThrow();
    c.setEmail(email);          // duplicate: unique constraint
  } catch (DataIntegrityViolationException e) {
    // never reached: nothing was sent yet
  }
}   // the error appears here, at commit
```

The SQL runs later

at flush or commit, not on the line that changed the object

try/catch misses it

the database error does not show up inside the try block

The fix

Call repository.flush() inside the try block when you need the error right there. The transaction is then rollback-only: report the error, do not carry on

In your daily job: find try/catch blocks around save() that expect a database error



TOPIC 1 – HOW TO PREVENT IT

Habits that avoid the surprises

One transaction per job

load, change and finish inside one @Transactional method

No save() on managed entities

dirty checking does it: save() is for new or detached entities

readOnly = true for reads

a read only transaction does not flush by accident

Flush when you need the error now

repository.flush() or saveAndFlush(...)

Return DTOs across layers

no detached entities and no lazy loading in the web layer

Make the SQL visible

spring.jpa.show-sql=true while you learn

EXERCISE – STARTER CODE

See the persistence context at work

CUSTOMERSERVICE.JAVA (STARTER)

```java
@Service
public class CustomerService {
  private final CustomerRepository repository;   // constructor omitted

  @Transactional
  public void rename(Long id, String newName) {
    // TODO step 1
  }

  public Customer load(Long id) {
    return repository.findById(id).orElseThrow();
  }

  @Transactional
  public void changeEmail(Long id, String email) {
    try {
      // TODO step 3
    } catch (DataIntegrityViolationException e) {
      // the console tells you that it was caught here
    }
  }
}
```

CustomerRepository is a plain JpaRepository<Customer, Long>. Before every test, Ann (customer 1, annId in the tests) and Bob (customer 2, bobId) are inserted. The email column is unique. SQL logging is on.



YOUR TASK

See the persistence context at work

1. Dirty checking: in rename(), load the customer and change its name, but do NOT call save(). Run the test: is the new name stored? Check the SQL in the console

2. Detached: no code needed. Run step2_detached_theChangeIsNotStored: it changes a customer outside a transaction and loads it again. Which name do you get, and why?

3. Flush timing: in changeEmail(), give customer 2 (Bob) the email of customer 1 (Ann) inside the try block. Where does the exception appear? Then add repository.flush() inside the try and run again

Bonus: In PersistenceBonusTest, make the change to the detached entity reach the database with one repository call. Which SQL do you see?



RECAP – TOPIC 1

Spring Data JPA & the Persistence Context

- **Managed inside a transaction, detached outside it**
  Entities you load or save in a transaction are tracked by the persistence context. Outside it, they are plain objects

- **Dirty checking writes your changes**
  A managed entity that you change is updated at commit, without a save() call

- **Flush is not commit**
  Flush sends the SQL and commit makes it permanent. Database errors show up at one of those two moments

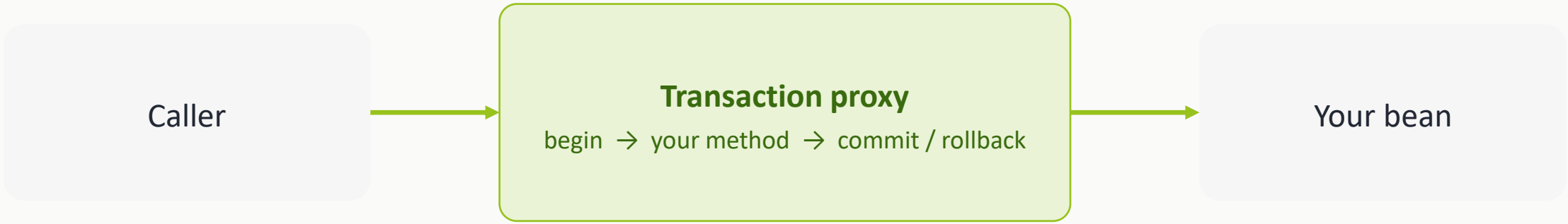
- **Keep the job in one transaction**
  Load, change and finish inside one @Transactional method, and return DTOs across layers



DAY 2 • TOPIC 2

Transactions in Spring Data JPA

Where we left off: @Transactional works through a proxy



All or nothing

either every change in the transaction is committed, or none of them

Only calls from outside pass the proxy

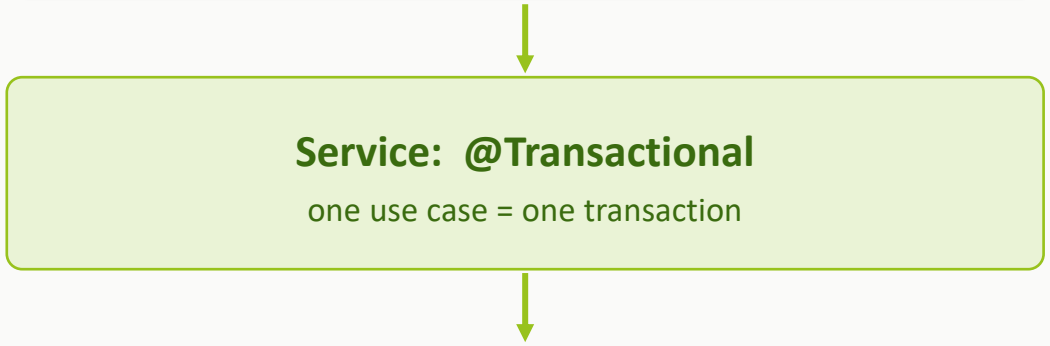
self-invocation and private methods are not transactional

The persistence context lives inside it

so the boundary decides when entities are managed and flushed

Where does the transaction start?

Controller: no transaction



Repository: joins the transaction

On the service method of one use case

for example placing an order, not on every method

Repositories are transactional too

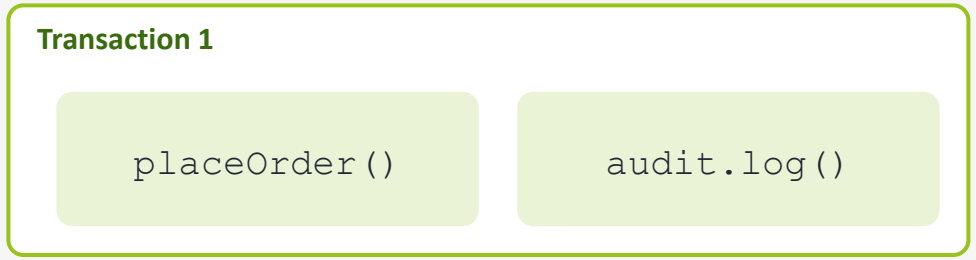
they join the transaction of the service, or run in their own when there is none

Class level or method level

the same rules apply, and the setting on the method wins

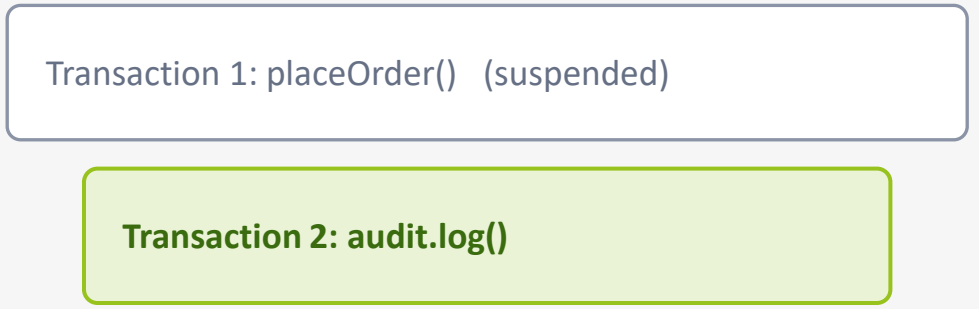
Propagation: join, or start a new transaction

REQUIRED (the default)



one transaction: both succeed, or both roll back

REQUIRES_NEW



the inner transaction commits on its own

MANDATORY
must already exist, otherwise an error

SUPPORTS
joins if there is one, runs without otherwise

NOT_SUPPORTED
suspends the transaction and runs without

NEVER
fails if a transaction exists

NESTED
a savepoint; usually not available with JPA

REQUIRES_NEW in practice

AUDITSERVICE.JAVA

```java
@Service
public class AuditService {

  @Transactional(propagation = Propagation.REQUIRES_NEW)
  public void log(String message) {
    audits.save(new AuditEntry(message));
  }   // commits here, whatever happens to the caller
}
```

Use it for

an audit log

a record of a failed attempt

work that must survive a rollback

Watch out: two transactions at once

while the inner one runs, the outer one is only suspended: it can still hold locks, and the inner one may need a second connection

Watch out: it needs the proxy

a REQUIRES_NEW method called with this is not a new transaction, so put it in a separate bean

In your daily job: search your code for REQUIRES_NEW and check why it is there



TOPIC 2 – WHAT IS IT

Rollback rules: which exceptions undo the transaction?

RuntimeException	ROLLBACK	an unchecked exception undoes the transaction
Error	ROLLBACK	undone as well
Checked exception	COMMIT	the transaction commits anyway, unless you add rollbackFor

TO ROLL BACK ON A CHECKED EXCEPTION TOO

```
@Transactional(rollbackFor = OrderRejectedException.class)
public void placeOrder(Order order) throws OrderRejectedException {
  // ...
```

an exception that you catch inside the method never reaches the proxy, so there is no rollback



TOPIC 2 – WHAT IS IT

Isolation: what do concurrent transactions see of each other?

	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible
SERIALIZABLE	prevented	prevented	prevented

Spring uses the default of your database unless you set isolation on @Transactional. Higher levels mean fewer surprises, but more locking and more failures. The details differ per database.

Dirty read

you read changes that another transaction has not committed yet

Non-repeatable read

the same row, read twice, has different values

Phantom read

the same query, run twice, returns different rows

A business transaction is more than a database transaction

One business transaction: place an order

1 Reserve stock
database transaction 1

→

2 Charge payment
another system: not part of any database transaction

→

3 Save the order
database transaction 2

→

4 Send confirmation
a message to another system

- A database transaction covers one database, over one connection
- A call to another system cannot be rolled back by it: you undo it with another call (compensation)
- So design the use case first, then decide where the transaction boundaries go

Keep transactions short

A TRANSACTION THAT IS TOO LONG

```
@Transactional
public void placeOrder(Order order) {
  repository.save(order);
  paymentClient.charge(order);          // remote HTTP call
  mailService.sendConfirmation(order); // can be slow
}
```

While it runs, it holds

a database connection and the locks on the changed rows

What goes wrong

slow calls pile up, the connection pool runs empty, other transactions wait for locks

Better

Do remote calls outside the transaction, and keep each transaction as small as possible: load, change, commit.

Process big batches in chunks, with one transaction per chunk.

In your daily job: look for HTTP or messaging calls inside @Transactional methods

The exception you caught, and the rollback you got

THE PROBLEM

```
@Transactional
public void placeOrder(Order order) {
  orders.save(order);
  try {
    stockService.reserve(order);   // @Transactional, throws
  } catch (RuntimeException e) {
    // handled: carry on without stock
  }
}   // commit fails: UnexpectedRollbackException
```

One transaction for both

reserve() joins the transaction of placeOrder()

A failure marks it rollback-only

catching the exception does not undo that mark

The fix

Decide who owns the failure: let the exception propagate, or give reserve() REQUIRES_NEW so that it fails on its own

In your daily job: look for try/catch blocks around calls to other @Transactional methods

The transaction that is not there

Self-invocation

a call via this never passes the proxy (Day 1)

Private methods

the proxy cannot intercept them, so the annotation is ignored

Not a Spring bean

an object created with new has no proxy at all

A swallowed exception

a catch block inside the method hides the failure from the proxy: the transaction commits

In your daily job: search your code for @Transactional on private methods



TOPIC 2 – HOW TO PREVENT IT

Habits for reliable transactions

One use case, one service method

put @Transactional where the use case starts

Keep it short

no remote calls and no waiting inside a transaction

Know the rollback rules

use rollbackFor for checked exceptions

Use REQUIRES_NEW on purpose

and know that it opens a second transaction

readOnly = true for reads

it documents the intent and avoids accidental flushes

Make transactions visible

set the log level of JpaTransactionManager to DEBUG

EXERCISE – STARTER CODE

Transactions in action

ORDERSERVICE.JAVA (STARTER)

```java
@Service
public class OrderService {
  private final OrderRepository orders;
  private final AuditService audit;

  @Transactional
  public void placeOrder(Order order) {
    audit.log("placing order");
    orders.save(order);
    if (order.getAmount() > 1000) {
      throw new IllegalStateException("Amount too high!");
    }
  }
}
```

AUDITSERVICE.JAVA (STARTER)

```java
@Service
public class AuditService {
  private final AuditRepository audits;
  private boolean failing;   // step 3

  @Transactional
  public void log(String message) {
    audits.save(new AuditEntry(message));
    if (failing) {
      throw new IllegalStateException(
          "Audit store unavailable");
    }
  }
}
```

OrderRepository and AuditRepository are plain Spring Data repositories. SQL and transaction logging are switched on. After every run, the test prints how many rows each table has. The test switches the audit on and off with setFailing().



YOUR TASK

Transactions in action

1. Place an order with amount 2000 and read the counts that the test prints. Are the order and the audit entry saved? Read the transaction log lines and explain what you see

2. Replace the IllegalStateException by the checked OrderRejectedException (declared with throws). Is the order saved? Then add rollbackFor to placeOrder() and run again

3. Make the audit fail (auditService.setFailing(true)) and catch the exception around audit.log() in placeOrder(). What happens when placeOrder() ends, and why? Then fix it with REQUIRES_NEW on log()

4. No new code: step 4 turns green with the REQUIRES_NEW of step 3. Read the test output: what is saved, and which transactions do you see in the log?

Bonus: Set propagation = Propagation.MANDATORY on AuditService.logInExistingTransaction() and call it from a test without a transaction. What do you get?



RECAP – TOPIC 2

Transactions in Spring Data JPA

- **The service method is the boundary**
  One use case is one transaction, started by the proxy on the service method

- **Propagation decides: join or start new**
  REQUIRED joins the existing transaction, REQUIRES_NEW starts a separate one that commits on its own

- **Only unchecked exceptions roll back**
  Checked exceptions commit, and a failing inner call marks the whole transaction rollback-only

- **Keep it short, and think bigger than one transaction**
  No remote calls inside a transaction, and a business transaction is more than one database transaction



DAY 2 • TOPIC 3

Data Consistency Issues in Practice



TOPIC 3 – BASICS

What does consistent data mean?

Consistent

the data satisfies the rules of the business, at every moment that someone can look at it

EXAMPLES OF RULES

- The stock of a product is never negative
- An order total equals the sum of its lines
- Every payment belongs to an order

WHO PROTECTS THEM?

The database

constraints: not null, unique, foreign key, check

The code

validation and business logic

The transaction

all or nothing



TOPIC 3 – WHAT IS IT

Three ways data becomes inconsistent

Two at once

several transactions change the same data at the same time: lost updates and double bookings

Cases 1 and 2

Two places

the same fact lives in two places that stop agreeing: a database and a message, memory and a row

Cases 3 and 4

Half done

a job stops halfway: a missing rollback, a swallowed exception, a transaction that never started

Topic 2

in the next four cases we look at what these look like in code



TOPIC 3 – CASE 1 OF 4

Two users, one customer

time →

Ann	loads customer 1	changes the email and saves	
Bob	loads customer 1		changes the phone number and saves

Result: Ann's new email is gone, the old one is back. Nobody got an error.

DISCUSS

- What does the customer look like at the end, and why?
- Which user will notice, and when?
- Would a higher isolation level solve this?

Last writer wins

What happened

Bob's UPDATE wrote all columns, using his old copy: the old email came back

Why there was no error

each save was valid on its own, and the database cannot know that Bob's copy was out of date

OPTIMISTIC LOCKING

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;

  @Version
  private Long version;
  // ...
}
```

How to avoid it

Add a @Version field: the UPDATE only succeeds when the version is unchanged, otherwise the second save fails and the user reloads.

Alternatives: a pessimistic lock, or an atomic UPDATE statement.

In your daily job: check which entities users can edit at the same time, and whether they have a @Version field



TOPIC 3 – CASE 2 OF 4

Check, then act

STOCKSERVICE.JAVA

```java
@Transactional
public void reserve(Long productId, int amount) {
  Stock stock = stocks.findByProductId(productId);
  if (stock.getQuantity() >= amount) {   // the check
    stock.setQuantity(stock.getQuantity() - amount);
  } else {
    throw new OutOfStockException();
  }
}
```

The situation

One item is left in stock.

Two customers order it at the same moment.

Both transactions run this method.

DISCUSS

What is the stock afterwards, and how many orders were accepted?

Does @Transactional protect us here?

How would you prevent it?



TOPIC 3 – CASE 2: THE ANSWER

The check is not the update

What happened

both read quantity 1, both passed the check, both wrote 0: two orders for one item

Why @Transactional does not help

it makes the steps atomic as a whole, but it does not stop another transaction from reading the same old value in between

ONE ATOMIC STATEMENT (SQL)

```
update stock
set quantity = quantity - :amount
where product_id = :id
  and quantity >= :amount
```

0 rows updated means: out of stock

Other options

a @Version field so that the second save fails, a pessimistic lock on the row, or a CHECK (quantity >= 0) constraint as the last line of defence



In your daily job: look for “if it exists, then save” and “if enough, then subtract” without a lock or constraint



TOPIC 3 – CASE 3 OF 4

The order and the message

ORDERSERVICE.JAVA

```java
@Transactional
public void placeOrder(Order order) {
  orders.save(order);
  rabbit.convertAndSend("orders", order.getId());
}   // the commit happens here, after the message was sent
```

The situation

The warehouse system listens to the queue, and loads the order by its id.

DISCUSS

What if the warehouse is faster than the commit?

What if the commit fails after the message was sent?

What if the broker is down?

TOPIC 3 – CASE 3: THE ANSWER

Two systems, no shared transaction

The message arrives first

the warehouse looks for an order that is not visible yet

The commit fails

the message is already out, for an order that never existed

The broker is down

the exception rolls the order back, or, if you catch it, the order exists without a message

A database transaction cannot include the broker: the two have no shared transaction

Where we go next

Topic 4: publish after the commit, and the outbox pattern

In your daily job: find places where a message is sent inside a transaction



TOPIC 3 – CASE 4 OF 4

The bulk update that gets overwritten

PRODUCTSERVICE.JAVA

```java
@Transactional
public void renameAndRaise(Long id) {
  // the product has price 100
  Product p = products.findById(id).orElseThrow();
  products.raiseAllPrices();     // bulk update: price * 1.1
  p.setName("Sale item");        // rename it
}
```

The situation

A bulk update query raises every price by 10 percent. The same method also renames the product it loaded before.

DISCUSS

What is the price of this product afterwards?

Which SQL does Hibernate send at commit?

What would you change?

The persistence context does not know

What happened

the bulk update ran in the database, but the managed product still holds price 100. At commit Hibernate writes all columns of the changed entity, including the old price

Why

bulk queries bypass the persistence context

THE FIX

```
@Modifying(clearAutomatically = true,
           flushAutomatically = true)
@Query("update Product p set p.price = p.price * 1.1")
void raiseAllPrices();
```

Watch out

after the clear, the entity you loaded earlier is detached: reload it before you change it

Other options

do the bulk update first and load the entities after it, write only the changed columns with @DynamicUpdate, or use separate transactions

In your daily job: search for @Modifying queries and check whether entities were loaded earlier in the same transaction



TOPIC 3 – HOW TO PREVENT IT

Four questions for every write

Which rule must always hold?

write it down: a rule that nobody wrote down is enforced nowhere

Who else can change this data at the same time?

other users, other instances of the application, batch jobs

What happens if the job stops halfway?

roll back, retry, or compensate

Where does the truth live?

the database enforces what must never happen, the code handles the rest



DISCUSSION

Where could this happen in your systems?

- Which data can two people change at the same time?
- Which jobs touch more than one table, or more than one system?
- What happens in your system when a job stops halfway?



RECAP – TOPIC 3

Data Consistency Issues in Practice

- **Consistent means that the rules always hold**
  The database enforces what must never happen, the code handles the rest

- **Two at once: use a version, a lock or one atomic statement**
  A transaction alone does not stop two users from overwriting each other

- **Two places have no shared transaction**
  A message and a database change can disagree: topic 4 looks at what to do

- **The persistence context can be stale**
  Bulk updates bypass it: clear the context, or do not mix them with entity changes



DAY 2 • TOPIC 4

Asynchronous Integration & Transactions

Synchronous and asynchronous integration

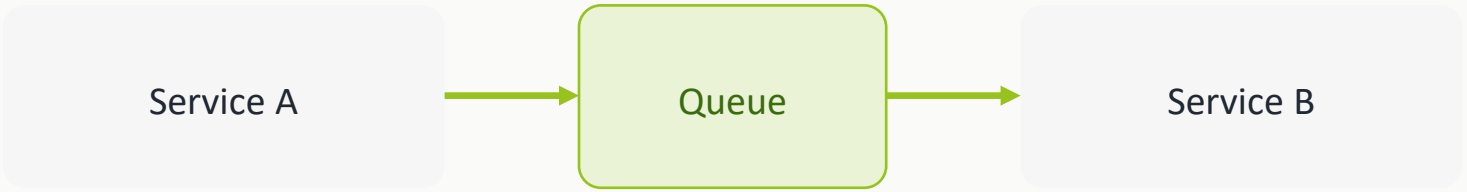
Synchronous

the caller waits for the answer



Asynchronous

the caller sends a message and carries on



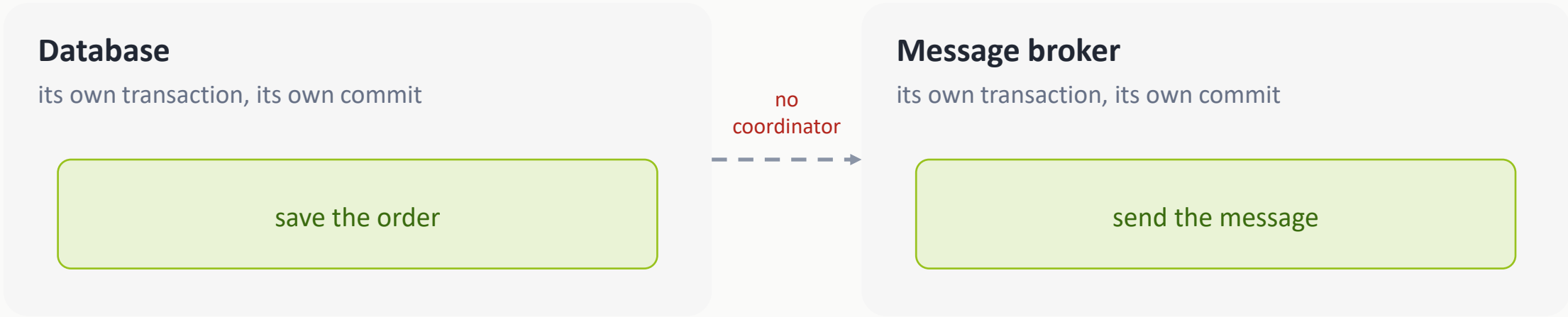
Why use it

decoupling in time, absorbing peaks, retrying later

What you give up

an immediate answer, a single transaction, and simple error handling

Why there is no shared transaction



- Each system commits on its own, at its own moment
- Distributed (XA) transactions exist, but RabbitMQ does not support them, and they are slow and complex
- So we do not make the two atomic: we make the failure harmless

Four outcomes of the same code

The commit and the send both succeed	**Fine**
The send succeeds, then the commit fails	**A message for an order that does not exist**
The consumer is faster than the commit	**The consumer cannot find the order yet**
The broker is down	**The order is rolled back, or, if you catch the exception, it exists without a message**

the same four lines of code, with four different outcomes

Fix 1: publish after the commit

PUBLISH AFTER THE COMMIT

```
@Transactional
public void placeOrder(Order order) {
  orders.save(order);
  events.publishEvent(new OrderPlaced(order.getId()));
}

@Component
class OrderMessagePublisher {
  @TransactionalEventListener    // default: AFTER_COMMIT
  void on(OrderPlaced event) {
    rabbit.convertAndSend("orders", event.orderId());
  }
}
```

Good

the message only leaves after a successful commit: no ghost messages

Still open

if the application crashes or the broker is down right after the commit, the message is lost

the event is a Spring event inside the application: the broker is only called after the commit

The gap after the commit

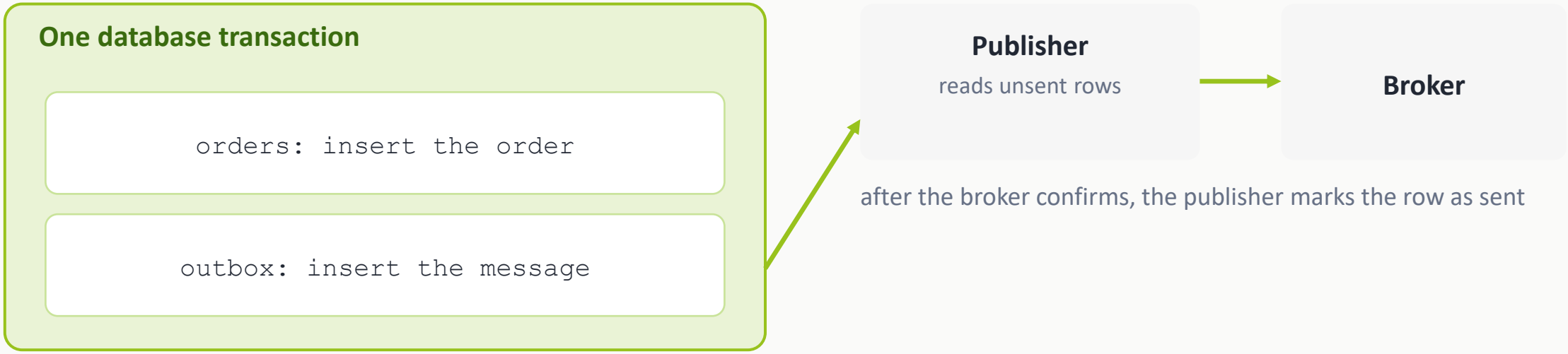


The order exists, but nobody was told, and nothing in the system notices

How would you ever find these orders?

In your daily job: check what happens in your system when the broker is down right after a commit

Fix 2: the transactional outbox



- The order and the message are saved together, or not at all
- If the publisher or the broker fails, the row stays unsent and is tried again: nothing is lost
- The price: a message can be sent twice, and it arrives a little later

The outbox in code

ORDERSERVICE.JAVA

```java
@Transactional
public void placeOrder(Order order) {
  orders.save(order);
  outbox.save(new OutboxMessage(
      "orders", order.getId()));
}   // one commit for both
```

OUTBOXPUBLISHER.JAVA

```java
@Scheduled(fixedDelay = 1000)
public void publishPending() {
  for (OutboxMessage m : outbox.findUnsent()) {
    try {
      rabbit.convertAndSend(m.getQueue(), m.getPayload());
    } catch (AmqpException e) {
      return;   // broker down: try again later
    }
    outbox.markSent(m.getId());   // own transaction
  }
}
```

Just a table

the outbox lives in the same database as the orders

In a real system

mark a row as sent only after the broker confirmed it

in the exercise project: broker.send(...) and BrokerUnavailableException

Delivery guarantees: what can a message promise?

At most once

send and forget: the message can be lost, but never duplicated

At least once

retry until it is confirmed: never lost, but possibly duplicated

Exactly once

not achievable between independent systems on its own: at least once plus an idempotent consumer gives the same effect

the outbox gives you at least once, which is the realistic choice

Idempotent consumers

ORDERLISTENER.JAVA

```java
// with Spring AMQP: @RabbitListener(queues = "orders")
@Transactional
public void handle(String messageId, Long orderId) {
  if (processed.existsById(messageId)) {
    return;                          // duplicate: ignore
  }
  warehouse.reserve(orderId);
  processed.save(new ProcessedMessage(messageId));
}
```

A unique message id

the same for every delivery of the same message

A primary key on it

if two copies arrive at once, the second insert fails and rolls back (case 2 again)

Idempotent

handling the same message a second time has no extra effect

In your daily job: check whether your message consumers can handle the same message twice



Pitfalls around publishing and consuming

No transaction, no event

a @TransactionalEventListener is ignored when no transaction is active, unless fallbackExecution is set

Database work after the commit

the transaction has already completed: writes in an AFTER_COMMIT listener need REQUIRES_NEW

Swallowed send failures

catching the broker exception hides that the message was never sent

Assuming exactly once

retries and redeliveries mean duplicates: design for them



TOPIC 4 – HOW TO PREVENT IT

Habits for asynchronous integration

Never send inside the writing transaction

publish after the commit, or use an outbox

Use an outbox when a lost message is not acceptable

and keep the publisher simple

A unique id in every message

the same for every delivery

Make consumers idempotent

a duplicate must be harmless

Design for eventual consistency

the other system is up to date a moment later

Monitor what is stuck

unsent outbox rows and the queue depth

EXERCISE – STARTER CODE

Messages and transactions

ORDERSERVICE.JAVA (STARTER)

```java
@Service
public class OrderService {
  private final OrderRepository orders;
  private final MessageBroker broker;

  @Transactional
  public void placeOrder(Order order) {
    orders.save(order);
    broker.send("orders", order.getId());   // the problem
    if (order.getAmount() > 1000) {
      throw new IllegalStateException("Amount too high!");
    }
  }
}
```

MESSAGEBROKER.JAVA (STARTER)

```java
public interface MessageBroker {
  void send(String queue, Long orderId);
  List<String> sentMessages();
  void setFailing(boolean failing);
}
```

A fake broker: it records what leaves the application, and setFailing(true) simulates a broker that is down.

After every run, the test prints the orders, the outbox rows and the messages that were sent. The starter project also contains the OrderPlaced event and an empty OrderMessagePublisher for step 2, and an OutboxMessage entity with an empty OutboxPublisher for step 4.



YOUR TASK

Messages and transactions

1. Place an order with amount 2000. Read what the test prints about the orders and the messages sent. What do you see, and what is wrong with it?

2. Move the sending to a @TransactionalEventListener (AFTER_COMMIT). Repeat with amount 2000 and with amount 50. When does the message leave in each case?

3. No new code: the test for this step turns green in step 2. Read its output: with a failing broker, what is the state of the order and of the message? How would you find such orders afterwards?

4. Replace the event by an outbox: save an OutboxMessage in the same transaction and let publishPending() send it. With the broker failing, what stays in the outbox? Then switch the broker on and publish again

Bonus: Deliver the same message twice to OrderListener. Make handle() idempotent: the ProcessedMessage table is ready, and its primary key is the message id.



RECAP – TOPIC 4

Asynchronous Integration & Transactions

- **A message and a database change have no shared transaction**
  Sending inside a transaction can give ghost messages and missing messages

- **Publish after the commit, or use an outbox**
  After the commit is simple but can lose a message, the outbox cannot

- **At least once is the realistic guarantee**
  So a message can arrive more than once

- **Make consumers idempotent**
  A unique message id and a primary key make duplicates harmless

RECAP – DAY 2

What we covered today

- **Spring Data JPA & the persistence context**
  Managed inside a transaction, detached outside it. Dirty checking writes your changes. Flush is not commit

- **Transactions in Spring Data JPA**
  The service method is the boundary. Propagation decides join or start new. Only unchecked exceptions roll back

- **Data consistency issues in practice**
  Use a version, a lock or one atomic statement. A bulk update bypasses the persistence context

- **Asynchronous integration & transactions**
  A message and a database change share no transaction: publish after the commit or use an outbox, and make consumers idempotent



Quiz

Questions?

Thank you for these two days

Your feedback

Scan the code below to fill in the Day 2 feedback form

